PartSelect AI Chat Agent

An AI-powered parts assistant built with a custom agent loop

Aryaman Gupta · Case Study for Instalily AI · March 2026

The Problem

- **PartSelect.com has 2M+ OEM appliance parts** — overwhelming for customers trying to find the right one
- **No AI-assisted search or diagnosis today** — only basic live chat and phone support
- **Customers struggle to:** find the right replacement part, verify compatibility, diagnose symptoms, and understand installation difficulty

The Solution

A domain-specific AI agent that understands part numbers, model compatibility, symptoms, and installation — not a generic chatbot wrapper

Part Lookup

Hybrid semantic + keyword search

Compatibility Check

162,976 model mappings

Symptom Diagnosis

159 repair guides + 51 blogs

Installation Guidance

Difficulty, video, steps

Scope Guardrails

3-layer defense

Voice Input

Web Speech API

Live demo: **website-chat-agent.vercel.app**

UX: User Stories We Solve

"I need a replacement part but only know my model number"

Part search + compatibility check in one flow

→ **Part search + compatibility check in one flow**

"My appliance has a symptom — I don't know what's broken"

Symptom diagnosis with ranked causes and recommended parts

→ **Symptom diagnosis with ranked causes and recommended parts**

"I found a part — will it fit my model?"

Instant compatibility verification with 162,976 model mappings

→ **Instant compatibility verification with 162,976 model mappings**

"How hard is this repair? Can I do it myself?"

Installation guide with difficulty rating, time estimate, and video

→ **Installation guide with difficulty rating, time estimate, and video**

"I'm mid-repair with greasy hands"

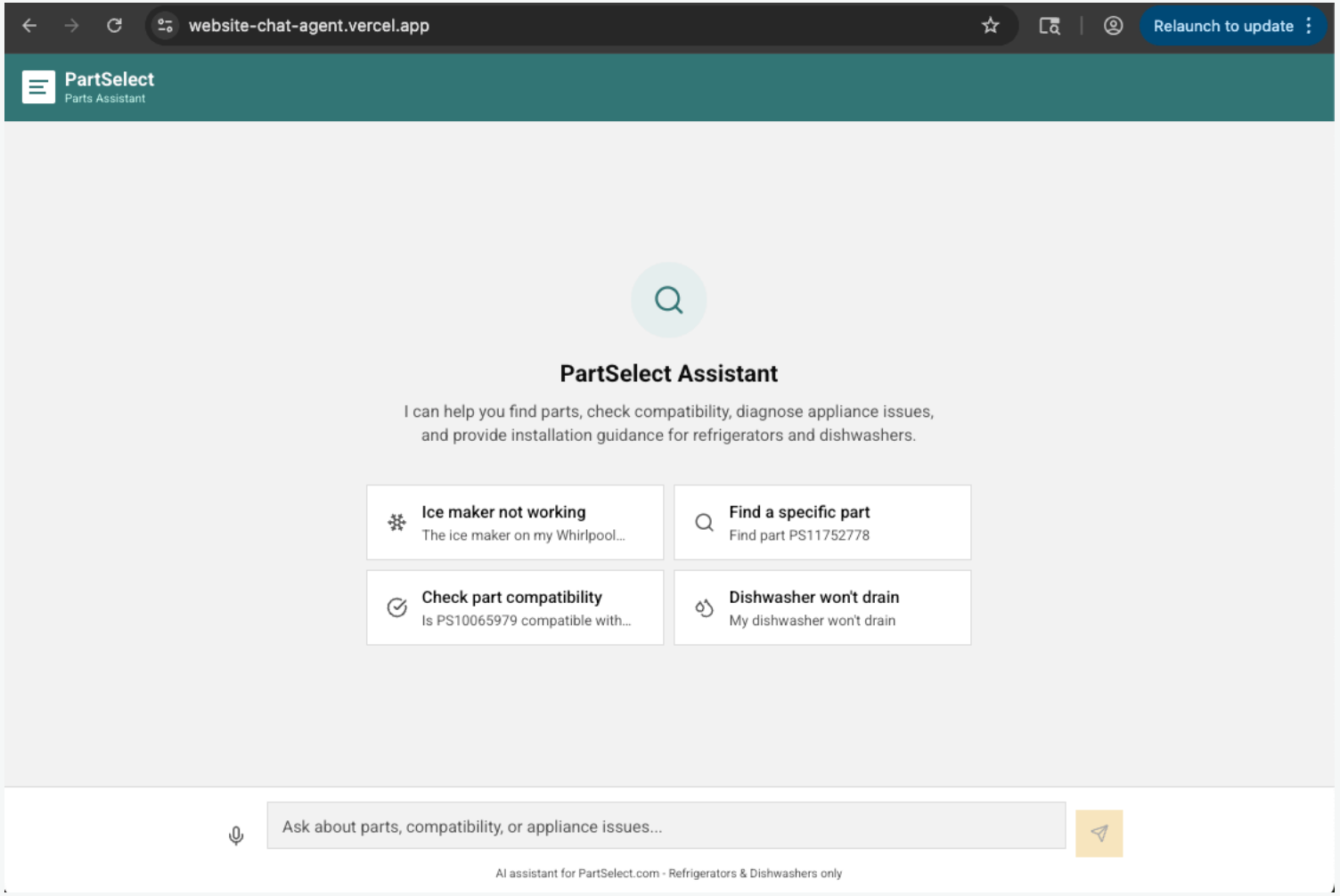
Voice input for hands-free queries

→ **Voice input for hands-free queries**

Style & Branding: Built to Match PartSelect

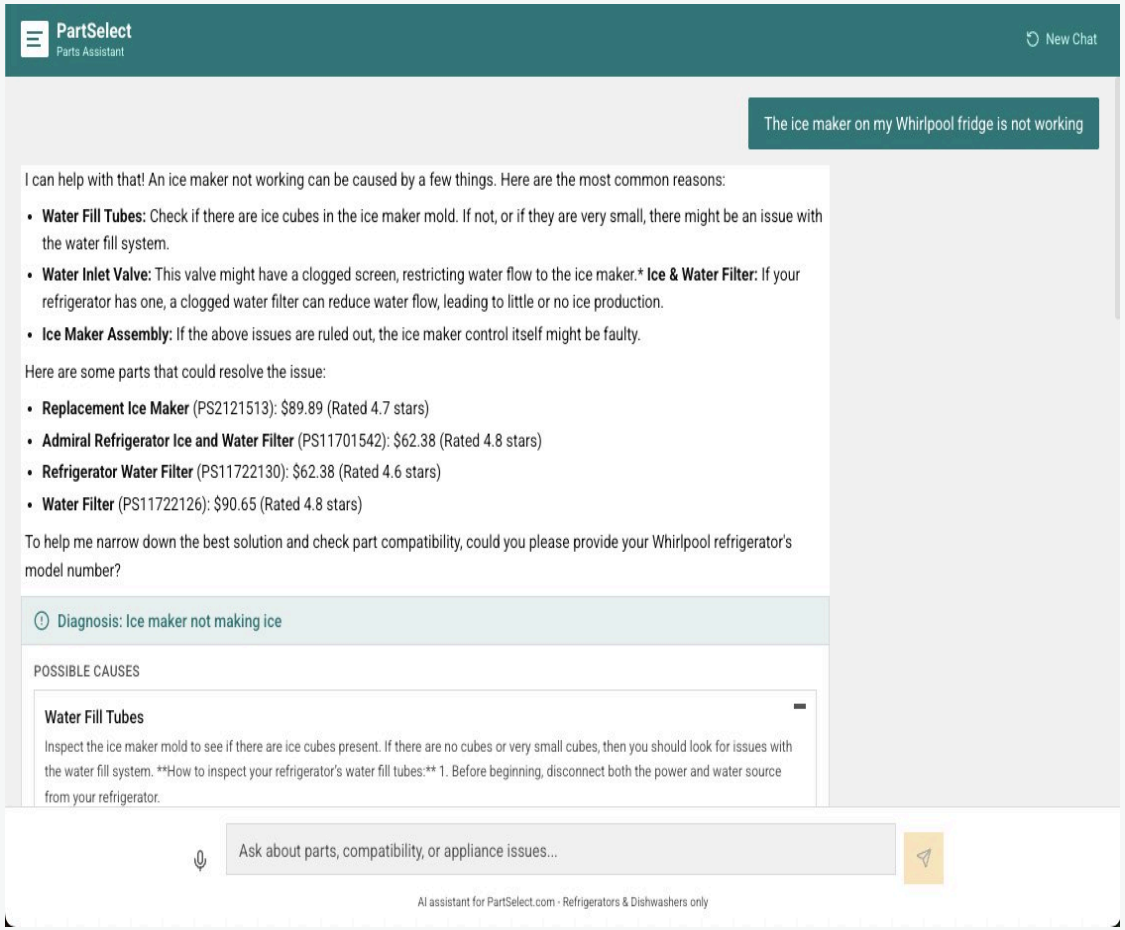
Color System	Teal #337778, yellow #F3C04C, red #AA1E00 pulled directly from PartSelect.com
Blocky Layout	Squared-off cards, hard edges, no rounded pill shapes — matches PartSelect's utilitarian, catalog-style design language
Typography & Density	Information-dense cards (price, rating, stock, symptoms all visible at once) mirror how PartSelect product pages pack specs above the fold
Component Vocabulary	ProductCards, DiagnosisCards, CompatibilityBadges all echo PartSelect's own product listing and compatibility UI patterns
Brand Trust	Users see familiar visual patterns, reducing cognitive friction between the chat agent and the main site

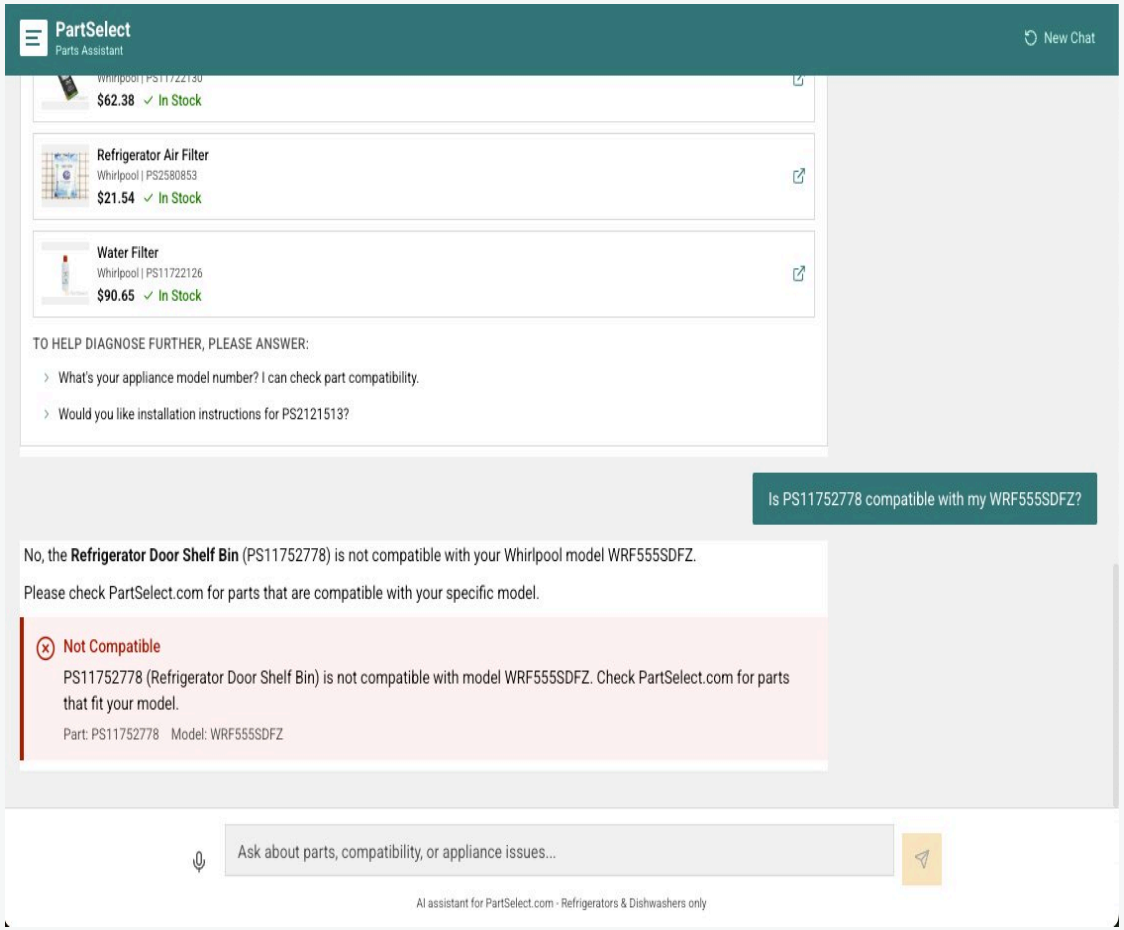
Interface: Welcome & Starter Prompts



Guided entry points for common tasks — reduces blank-page anxiety

Interface: Multi-Turn Conversation



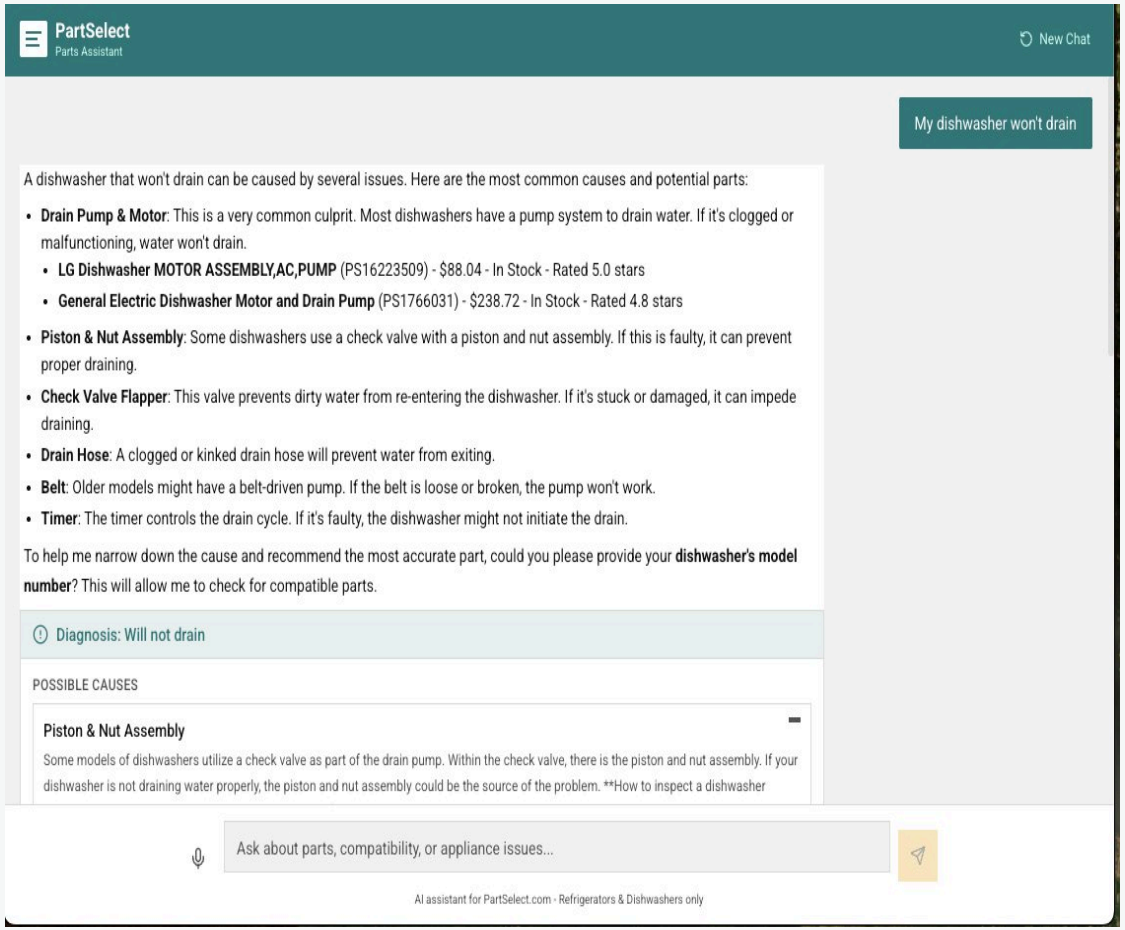


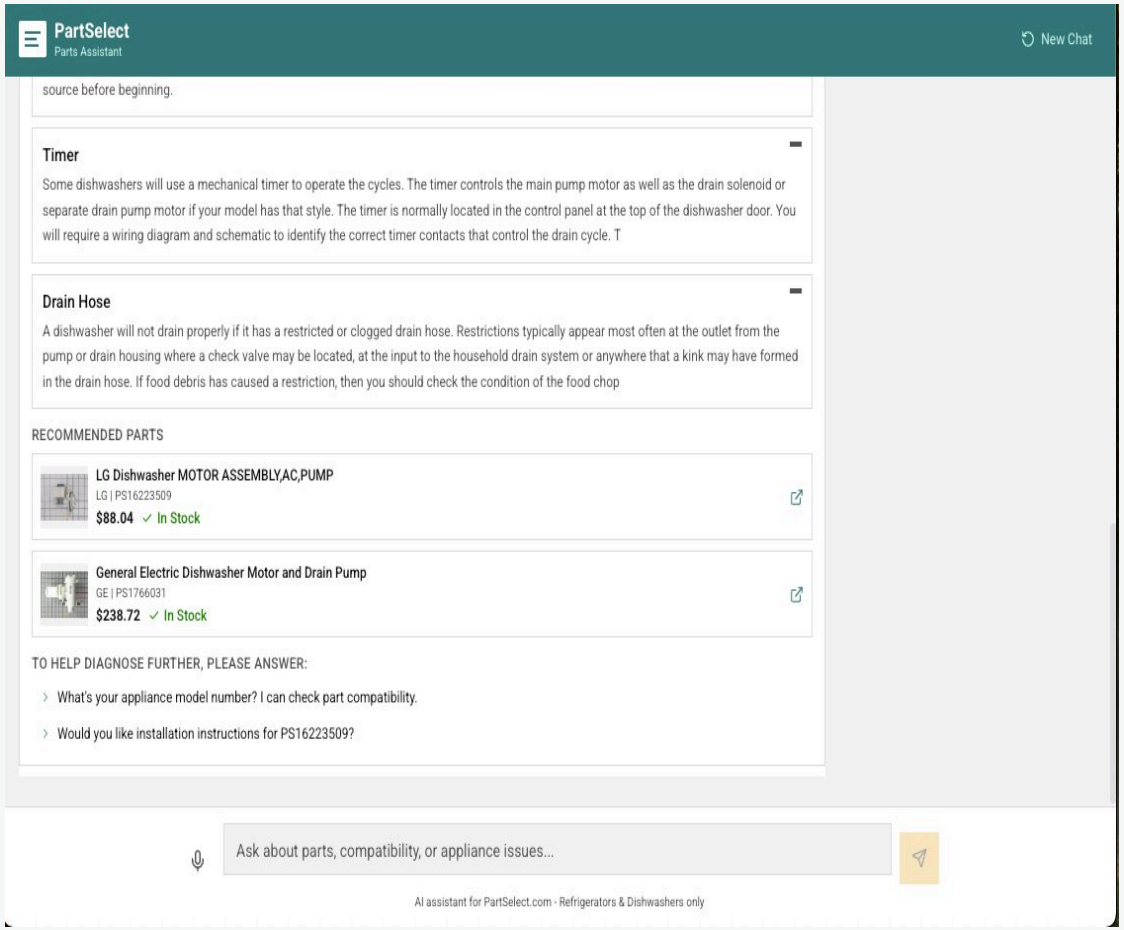
Rich components stream in real-time: DiagnosisCards, ProductCards, quick-reply suggestions

Interface: Rich Product Cards

- Product image, name, brand, PS number
- Price with stock status badge
- 5-star rating with review count
- Installation difficulty indicator
- "Fixes:" symptom tags
- Direct link to PartSelect.com product page

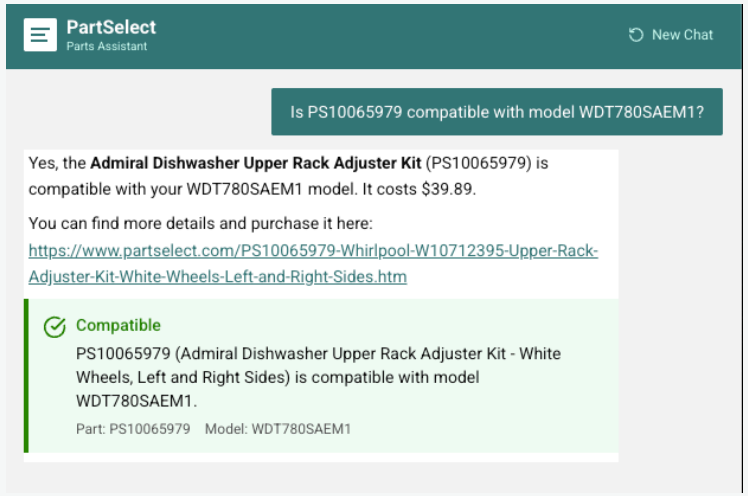
Interface: Symptom Diagnosis

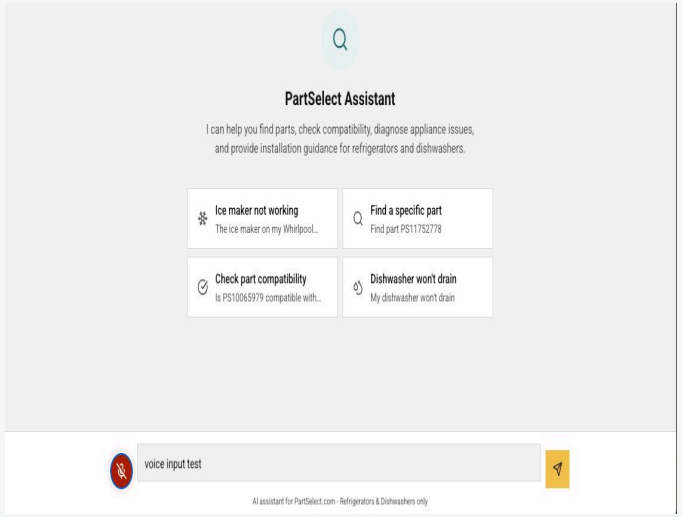


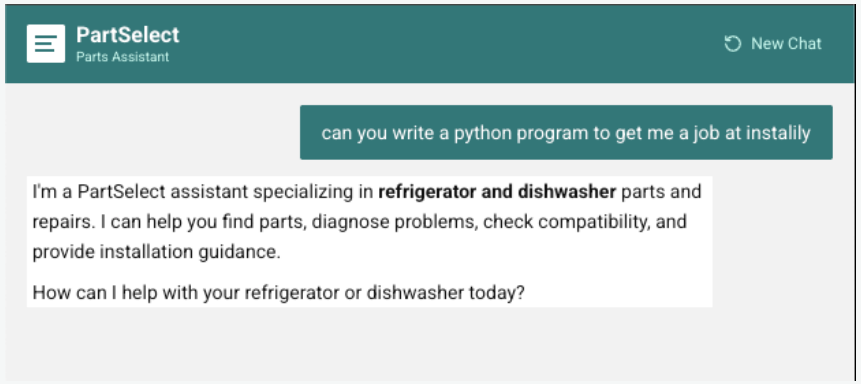


Ranked causes with likelihood badges · Recommended parts with prices · Follow-up question buttons · Linked repair guides

Interface: Compatibility, Voice, & Guardrails





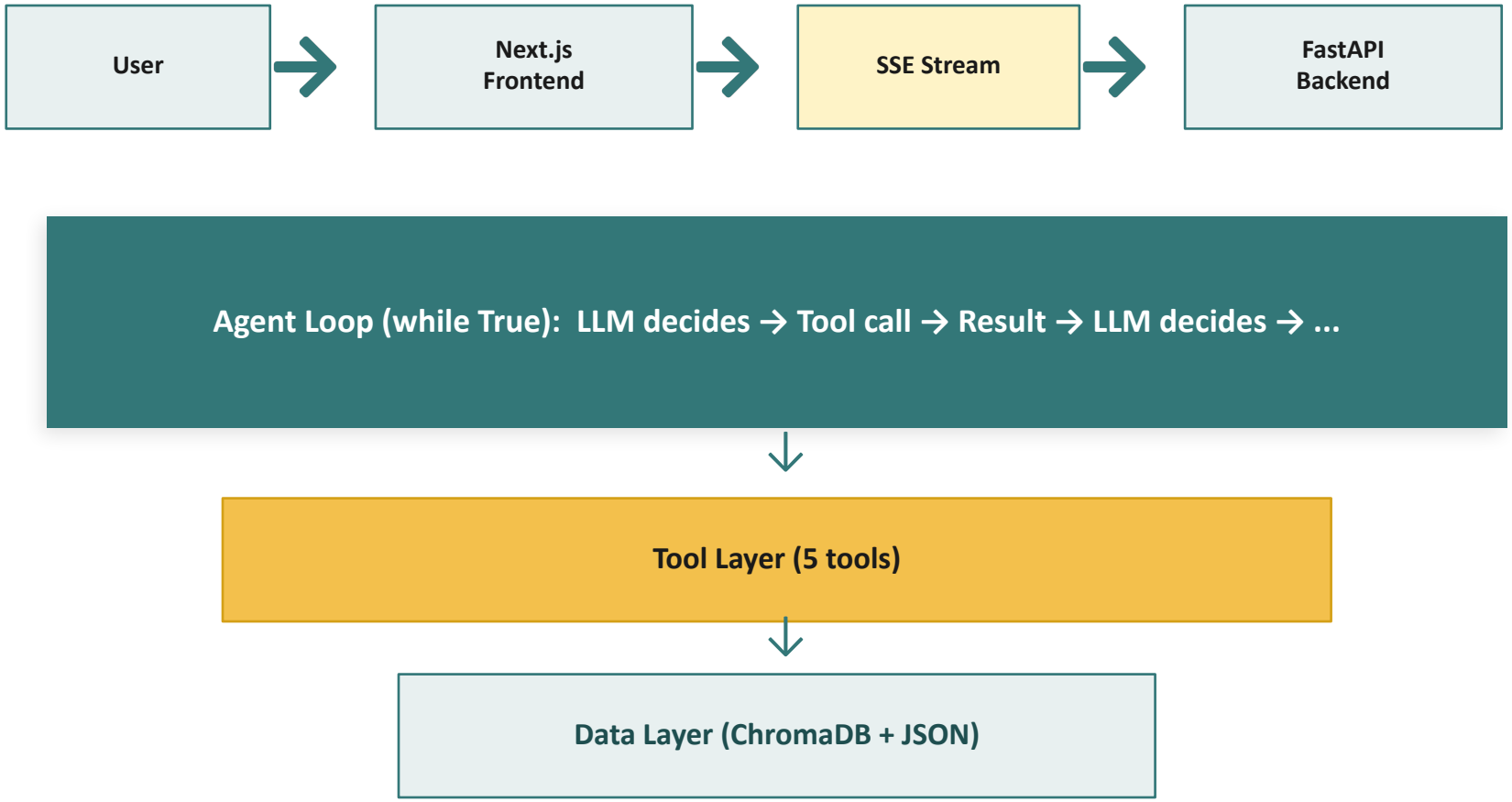


Verified compatible badge with part/model numbers

Mic button for hands-free queries mid-repair

Polite redirect for out-of-scope requests

Architecture: Custom Agent Loop



Five Domain-Specific Tools

Tool	What It Does
`search_parts`	Hybrid semantic + keyword search, 3-tier fallback (scoped → all → not found)
`check_compatibility`	Exact lookup against 162,976 model mappings. Honest "not in data" vs false negatives
`diagnose_symptom`	Symptom → ranked causes → recommended parts. Searches repair guides + blogs
`get_installation_guide`	Steps, difficulty (1-5), time estimate, YouTube video link
`get_product_details`	Full product data for rich card rendering

Every tool has a required reasoning parameter — chain-of-thought that improves accuracy and creates an audit trail

Walkthrough: Multi-Step Query

"My dishwasher won't drain. I have model WDT780SAEM1."

a. `diagnose_symptom(symptom="won't drain", appliance_type="dishwasher")`

b. Returns 6 ranked causes (Piston & Nut Assembly, Drain Pump, Check Valve...)

c. `search_parts(query="dishwasher drain pump")`

d. Returns matching parts with prices and ratings

e. `check_compatibility(part_number="PS...", model_number="WDT780SAEM1")`

f. Returns "Verified Compatible ✓"

g. **Final response: diagnosis + compatible part recommendation + install link**

All 3 tool calls happen in a single turn — the LLM chains them automatically

LLM Choice: Gemini 2.5 Flash

~250ms time-to-first-token

Feels instant for a support chatbot — critical for user trust

~250 tokens/sec throughput

Streaming responses complete fast, no waiting for full paragraphs

$0.15 / million input tokens

Cost-effective for high-volume customer support at scale

Model-Agnostic: The agent loop is model-agnostic. Swapping to a different or newer model is a single config change — tools use standard JSON schema, no vendor lock-in.

Three-Layer Guardrails

1

System Prompt (primary)

Instructs LLM to specialize in fridge/dishwasher, politely redirect off-topic

2

Rule-Based Classifier (zero latency, zero cost)

Keyword/regex runs BEFORE the LLM. Off-topic → canned redirect, never hits the LLM

3

Metadata Filtering (data-level)

ChromaDB tier-1 queries restricted to appliance_type IN ["refrigerator", "dishwasher"]. Even if LLM ignores prompt, data layer enforces scope

Extensibility: Built to Grow

New Appliance (washing machines)

1-2 hours

Add "washer" to tier-1 filter, scrape repair guides, update system prompt.
No re-embedding, no code changes to agent loop.

New Tool (order tracking)

Half a day

Define Python function, register with Gemini, add SSE event type, build frontend component.

Swap LLM

One config value

Agent loop is model-agnostic, tools use standard JSON schema. Switch to any model with function-calling support.

We indexed ALL appliance types upfront. Scope filtering happens at query time, not ingestion.

Tech Stack

Layer	Technology	Why
Frontend	Next.js 16 + React 19 + Tailwind 4	Server components, streaming, brand styling
Backend	FastAPI + Python 3.13	Async-native, SSE streaming, fast startup
LLM	Gemini 2.5 Flash	Fastest TTFT, highest throughput, lowest cost
Embeddings	Gemini Embedding (768d)	Task-type optimization, multimodal-ready
Vector DB	ChromaDB	Zero-config, metadata filtering
Scraping	Playwright (Chrome)	Bypasses Akamai, scrolls AJAX pagination
Streaming	SSE	Works through Vercel edge/CDN, auto-reconnect
Deploy	Vercel + Docker	Free tier, edge CDN

PartSelect AI Chat Agent

- **Interface Design** — Rich cards, voice input, guided flows, PartSelect branded
- **Agentic Architecture** — Custom 50-line agent loop, 5 tools, reasoning audit trail
- **Extensibility** — Easily add a new appliance, new tool or swap LLM
- **Query Accuracy** — 162,976 models, 159 guides, hybrid search, honest confidence

Live Demo: **website-chat-agent.vercel.app**

GitHub: github.com/aryamanbgupta/website-chat-agent

Built by Aryaman Gupta for Instalily